



CAL POLY

BUS 393 - Database Systems in Business

Airline Booking Platform - Database Design & Implementation

Arturo Ordaz-Gutiérrez, Christina Balingit, Maya V. Karl, Marshall Moore, Courtney Yuen

About Our Design

Airline Booking Platform

A centralized system that allows customers to search, book, and manage flights across multiple airlines and routes.

The platform manages the full booking process from start to finish:

- A customer creates a booking
- Passengers are assigned to flights
- Seats and luggage are added
- Payment is recorded

Database scope:

- 13 tables covering customers, airlines, airports, routes, flights, bookings, passengers, payments, luggage, and seat assignments
- Focused specifically on the booking and reservation process

Why a Relational Data...

01

Centralize all booking data; customers, flights, passengers, and payments; in one structured system

02

Enforce data integrity through relationships, foreign keys, and constraints so records stay accurate and consistent

03

Enable meaningful business queries; revenue by route, top airlines, customer spending history, and passenger tracking

Business Rules

Transaction Rules

- A booking generates exactly one payment
- A booking can include many flights and many passengers through booking_flight
- Each booking_flight links exactly one booking, one flight, and one passenger; one ticket
- A booking_flight can have many seat assignments and many luggage add-ons

Entity Rules

- A customer can make many bookings but each booking belongs to exactly one customer
- A customer can refer other customers through a self-referencing relationship
- A flight is operated by exactly one airline and follows exactly one route
- A route connects exactly one origin airport to exactly one destination airport
- A passenger must be exactly one type: adult, minor, or employee; never more than one
- A minor passenger must have a guardian who is an adult passenger
- An employee passenger is a crew member; they are not a customer

Conceptual Design

Customer

A customer is a person who registers on the platform to search and book flights. Customers provide contact information and can refer other customers through the platform's referral program. Attributes: customer_id (PK), first_name, last_name, email, phone, referred_by (FK - unary)

Airline

An airline is a company registered on the platform that operates one or more flights. Airlines are identified by a unique code and name. Attributes: airline_id (PK), airline_name, airline_code

Airport

An airport is a physical location that serves as either an origin or a destination for a route. The same airport can appear in many routes in either role. Attributes: airport_id (PK), airport_code, airport_name, city, state

Conceptual Design

Route

A route is a pairing of an origin airport and a destination airport. It captures the physical path a flight travels and stores the distance between the two airports. The same airport can serve as the origin or destination of many routes. Attributes: route_id (PK), origin_airport_id (FK), destination_airport_id (FK), distance_miles

Flight

A flight is a specific scheduled trip operated by an airline along a route. It captures departure and arrival times along with a base price. One airline can operate many flights and one route can have many flights scheduled on it. Attributes: flight_id (PK), airline_id (FK), route_id (FK), flight_number, departure_time, arrival_time, base_price

Booking

A booking is a reservation transaction made by a customer on the platform. It records when the reservation was made and its current status. One customer can make many bookings but each booking belongs to exactly one customer. Attributes: booking_id (PK), customer_id (FK), booking_date, booking_status

Conceptual Design

Passenger

A passenger is any person traveling on a flight. Every passenger must belong to exactly one of three disjoint subtypes: adult, minor, or employee. Paying passengers are linked to a customer but employee passengers are not. Attributes: passenger_id (PK), customer_id (FK - nullable), first_name, last_name, passport_number, passenger_type

Booking Flight

A booking_flight is a ternary associative entity that links exactly one booking, one flight, and one passenger; representing one ticket. It also serves as the anchor point for seat assignments and luggage add-ons. Attributes: booking_flight_id (PK), booking_id (FK), flight_id (FK), passenger_id (FK), ticket_price

Conceptual Design

Payment

A payment records the financial transaction associated with a booking. Each booking generates exactly one payment which captures the method, amount, date, and status of the transaction.

Attributes: payment_id (PK), booking_id (FK), payment_date, payment_method, amount_paid, payment_status

Luggage/Seat Assignment

A luggage record stores add-on baggage fees for a specific booking_flight. A seat assignment records the seat number, cabin class, and seat fee for a specific booking_flight.

Luggage Attributes: luggage_id (PK), booking_flight_id (FK), luggage_type, quantity, fee

Seat Assignment Attributes: seat_assignment_id (PK), booking_flight_id (FK), seat_number, cabin_class, seat_fee

Disjoint Subtype

Passenger

ADULT_PASSENGER

Paying adult traveler

Adds: date_of_birth,
frequent_flyer_number

MINOR_PASSENGER

Child traveler

Adds: date_of_birth,
guardian_passenger_id
(FK - ADULT_PASSENGER)
A minor must have an adult
guardian

EMPLOYEE_PASSENGER

Crew member; pilot, co-pilot, flight
attendant

Adds: employee_number,
job_title, hire_date, airline_id
(FK - AIRLINE) customer_id is
NULL - employees are not
customers

Special Relationships

Unary - CUSTOMER

The referred_by column on the CUSTOMER table is a foreign key that points back to customer_id on the same table

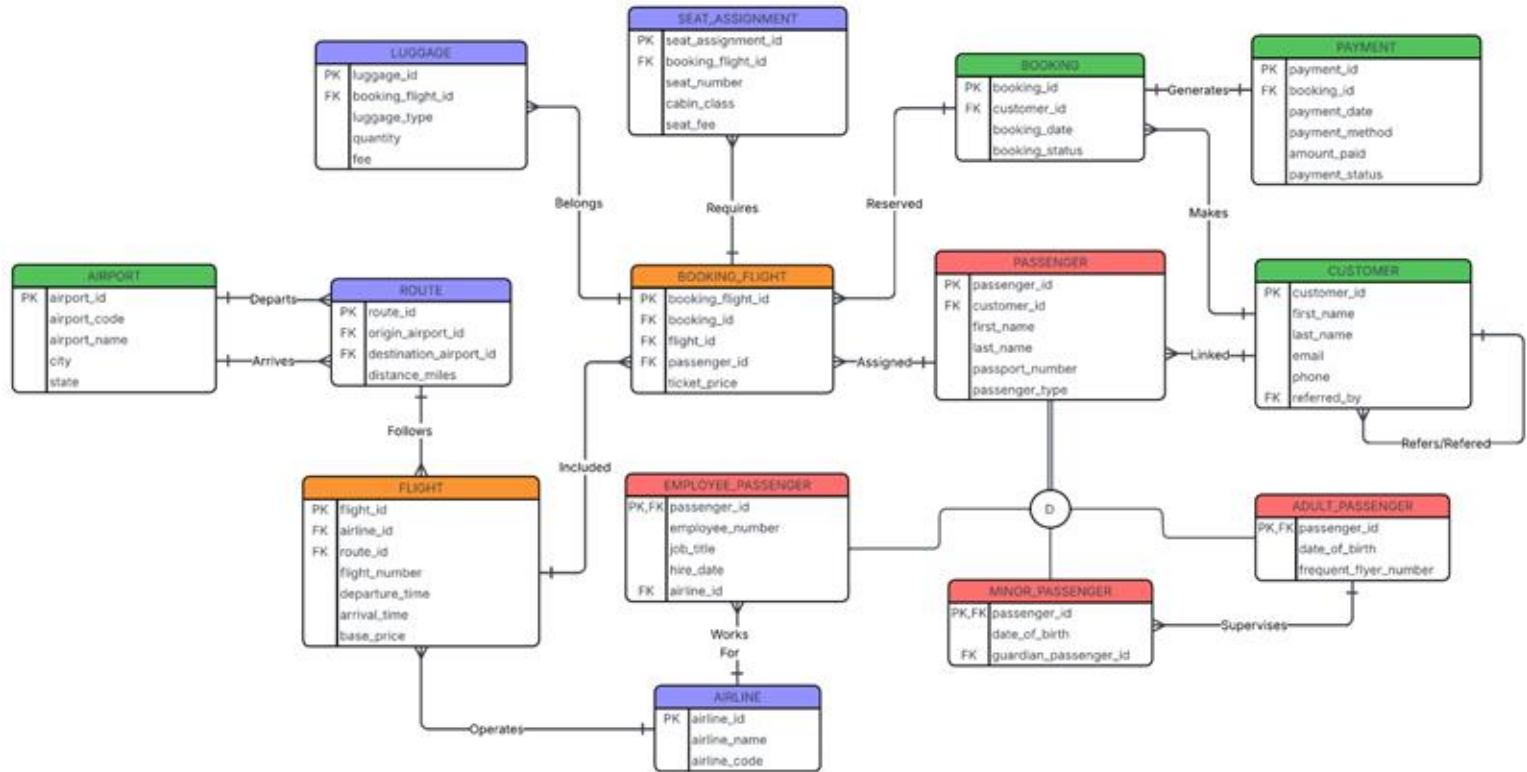
- A customer can refer many other customers to the platform
- NULL means the customer was not referred by anyone
- Shown in the ERD as a loop line from CUSTOMER back to itself

Ternary - BOOKING_FLIGHT

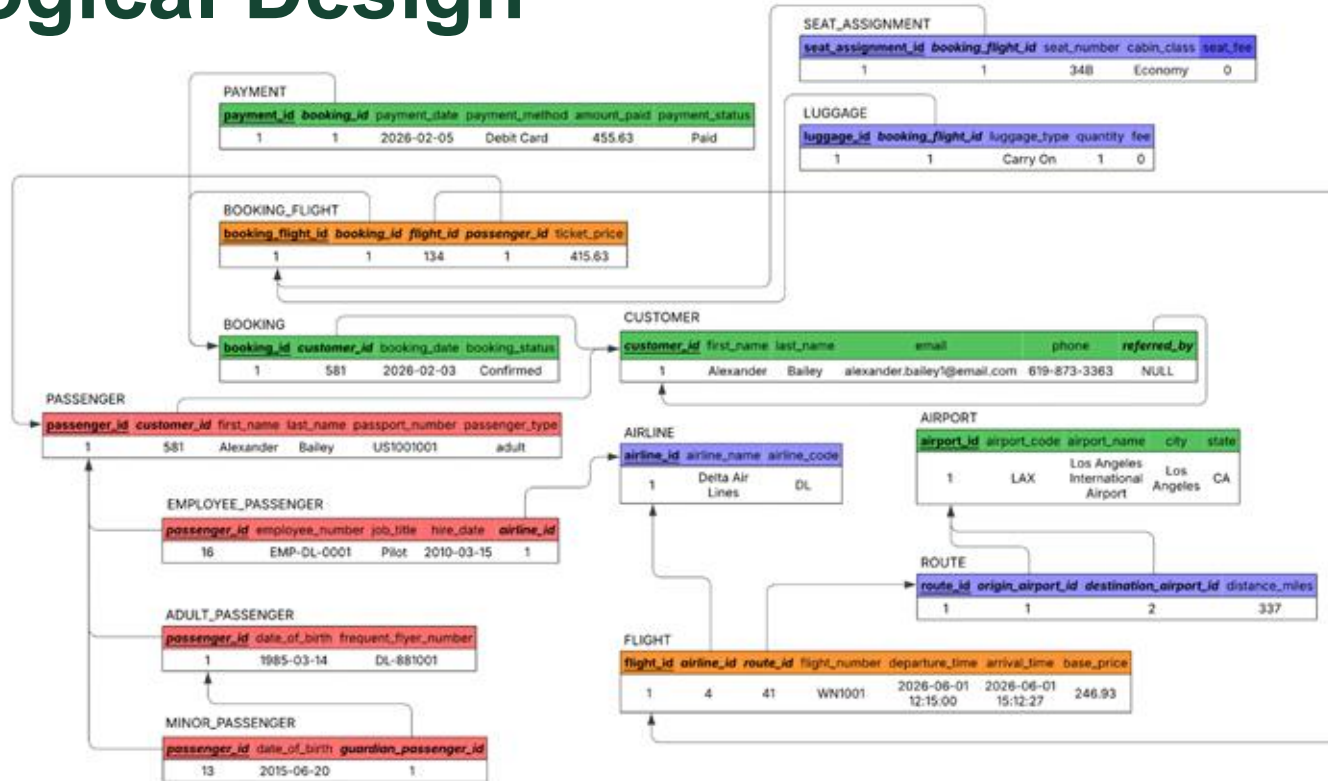
BOOKING_FLIGHT connects three entities: BOOKING, FLIGHT, and PASSENGER

- Each row represents one ticket; a specific passenger on a specific flight under a specific booking
- Cannot be split into binary pairs without losing information
- Carries its own attribute: ticket_price
- SEAT_ASSIGNMENT and LUGGAGE both anchor to BOOKING_FLIGHT rather than directly to FLIGHT or PASSENGER

Entity-Relationship Diagram



Logical Design



Logical Design

CUSTOMER					
<u>customer_id</u>	first_name	last_name	email	phone	<i>referred_by</i>
Foreign Key (referred_by) references customer (customer_id) - unary self-reference					

AIRLINE		
<u>airline_id</u>	airline_name	airline_code

AIRPORT				
<u>airport_id</u>	airport_code	airport_name	city	state

ROUTE			
<u>route_id</u>	<i>origin_airport_id</i>	<i>destination_airport_id</i>	distance_miles
Foreign Key (origin_airport_id) references airport (airport_id) Foreign Key (destination_airport_id) references airport (airport_id)			

Logical Design

FLIGHT						
<u>flight_id</u>	<i>airline_id</i>	<i>route_id</i>	flight_number	departure_time	arrival_time	base_price
Foreign Key (airline_id) references airline (airline_id) Foreign Key (route_id) references route (route_id)						

BOOKING			
<u>booking_id</u>	<i>customer_id</i>	booking_date	booking_status
Foreign Key (customer_id) references customer (customer_id)			

PASSENGER					
<u>passenger_id</u>	<i>customer_id</i>	first_name	last_name	passport_number	passenger_type
Foreign Key (customer_id) references customer (customer_id); NULL for employees					

Logical Design

ADULT_PASSENGER				
<u><i>passenger_id</i></u>	date_of_birth	frequent_flyer_number		

MINOR_PASSENGER				
<u><i>passenger_id</i></u>	date_of_birth	<i>guardian_passenger_id</i>		
Foreign Key (guardian_passenger_id) references adult_passenger (passenger_id)				

EMPLOYEE_PASSENGER				
<u><i>passenger_id</i></u>	employee_number	job_title	hire_date	<i>airline_id</i>
Foreign Key (airline_id) references airline (airline_id)				

Booking Flight; Ternary Associative Entity				
<u><i>booking_flight_id</i></u>	<i>booking_id</i>	<i>flight_it</i>	<i>passenger_id</i>	ticket_price
Foreign Key (booking_id) references booking (booking_id) Foreign Key (flight_id) references flight (flight_id) Foreign Key (passenger_id) references passenger (passenger_id)				

Logical Design

PAYMENT

<u>payment_id</u>	<i>booking_id</i>	payment_date	payment_method	amount_paid	payment_status
-------------------	-------------------	--------------	----------------	-------------	----------------

Foreign Key (booking_id) references booking (booking_id)

LUGGAGE

<u>luggage_id</u>	<i>booking_flight_id</i>	luggage_type	quantity	fee
-------------------	--------------------------	--------------	----------	-----

Foreign Key (booking_flight_id) references booking_flight (booking_flight_id)

SEAT_ASSIGNMENT

<u>seat_assignment_id</u>	<i>booking_flight_id</i>	seat_number	cabin_class	seat_fee
---------------------------	--------------------------	-------------	-------------	----------

Foreign Key (booking_flight_id) references booking_flight (booking_flight_id)

Implementation

- **VARCHAR** and **DATE** for names, emails, and dates to accommodate varying lengths of text
- **DECIMAL(10, 2)** for all monetary values; ticket prices, fees, and payments
- **AUTO_INCREMENT** for all primary keys to ensure unique identifiers
- **CHECK** constraint on **passenger_type** to enforce the disjoint subtype rule
- **DEFAULT NULL** on **referred_by** and **customer_id** for optional foreign keys

```
CREATE TABLE customer (  
    customer_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    gmail VARCHAR(100) NOT NULL UNIQUE,  
    phone VARCHAR(20),  
    referred_by INT DEFAULT NULL,  
    FOREIGN KEY (referred_by) REFERENCES customer(customer_id)  
);
```

Database Demonstration

Tables

Use `DESCRIBE table_name`: to return description of tables and their attributes

Field	Type	Null	Key	Default	Extra
flight_id	int	NO	PRI	NULL	auto_increment
airline_id	int	NO	MUL	NULL	
route_id	int	NO	MUL	NULL	
flight_number	varchar(20)	NO		NULL	
departure_time	datetime	NO		NULL	
arrival_time	datetime	NO		NULL	
base_price	decimal(10,2)	NO		NULL	

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	auto_increment
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	NO		NULL	
gmail	varchar(100)	NO	UNI	NULL	
phone	varchar(20)	YES		NULL	
referred_by	int	YES	MUL	NULL	

Row

Use `SELECT * FROM table_name LIMIT 10`; to return 10 rows from specific table

Customer Table:

	customer_id	first_name	last_name	gmail	phone	referred_by
	1	Alexander	Bailey	alexander.bailey1@email.com	619-873-3363	NULL
	2	Ronald	Cox	ronald.cox2@email.com	702-612-1659	NULL
	3	Barbara	Jones	barbara.jones3@email.com	415-830-6456	NULL
	4	Megan	Gutierrez	megan.gutierrez4@email.com	206-252-9446	NULL
	5	Emily	Adams	emily.adams5@email.com	619-266-7438	1
	6	Scott	Ramos	scott.ramos6@email.com	619-619-9351	NULL
	7	Camila	Price	camila.price7@email.com	213-586-1272	NULL
	8	Jack	Nelson	jack.nelson8@email.com	310-526-6040	NULL
	9	Michael	Kim	michael.kim9@email.com	206-371-4728	NULL
	10	Susan	Howard	susan.howard10@email.com	206-274-9589	NULL

Implementation - Disjoint

- `passenger_type` constrained to exactly three values using a CHECK constraint
- Each subtype table uses `passenger_id` as both **PRIMARY KEY** and **FOREIGN KEY** back to the passenger supertype
- `customer_id` set to **DEFAULT NULL** to allow employee passengers who are not customers
- `guardian_passenger_id` on `minor_passenger` references `adult_passenger`; enforcing that every minor has an adult guardian

```
CREATE TABLE passenger (  
    passenger_id INT PRIMARY KEY AUTO_INCREMENT,  
    customer_id INT DEFAULT NULL,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    passport_number VARCHAR(30),  
    passenger_type VARCHAR(10) NOT NULL CHECK (passenger_type IN ('adult', 'minor', 'employee')),  
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)  
);
```

Database Demonstration - Disjoint

Passenger Table:

passenger_id	customer_id	first_name	last_name	passport_num...	passenger_type
1	581	Alexander	Bailey	US1001001	adult
2	729	Ronald	Cox	US1002002	adult
3	943	Barbara	Jones	US1003003	adult
4	347	Megan	Gutierrez	US1004004	adult
5	1000	Emily	Adams	US1005005	adult

Minor Passenger Table:

passenger_id	date_of_bir...	guardian_passenger...
13	2015-06-20	1
14	2017-11-03	2
15	2019-04-12	6

Data Population - Python CSV Parser

- Data is stored in 14 separate CSV files; one per table; inside a **DATA/** folder
- A Python script (**load_data.py**) connects to MySQL, reads each CSV, and loads the data automatically
- **INSERT IGNORE** prevents duplicate key errors when the script is re-run
- Text values of “**NULL**” in CSVs are converted to real SQL **NULL**; handles nullable foreign keys like **referred_by** and **customer_id**
- Foreign key checks are disabled during loading and re-enabled after to allow flexible load order

```
import csv
import mysql.connector
import os
import re

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": ""
}

DATABASE = "airline_booking"
SQL_FILE = os.path.join(os.path.dirname(__file__), "airline_booking.sql")
DATA_DIR = os.path.join(os.path.dirname(__file__), "..", "Data")

# CSV filename => table name (only where they differ)
TABLE_MAP = {"seat_assignments.csv": "seat_assignment"}

# CSV column => table column (only where they differ)
LUMP_MAP = {"customer.csv": ("email": "gmail")

# Order respects FK dependencies
AD_ORDER = [
    "customer.csv",
    "airline.csv",
    "airport.csv",
    "route.csv",
    "flight.csv",
    "booking.csv",
    "passenger.csv",
    "adult_passenger.csv",
    "minor_passenger.csv",
    "employed_passenger.csv",
    "booking_flight.csv",
    "payment.csv",
    "luggage.csv",
    "seat_assignments.csv",
]

def extract_create_statements(path):
    with open(path, "r") as f:
        content = f.read()
        idx = content.find("INSERT INTO")
        if idx != -1:
            content = content[idx:]
            stmts = []
            for m in re.finditer("CREATE TABLE.*?", content, re.DOTALL):
                stmts.append(m.group())
            return stmts

def load_csv(cursor, csv_file, table_name, col_map):
    path = os.path.join(DATA_DIR, csv_file)
    with open(path, "r") as f:
        reader = csv.reader(f)
        headers = next(reader)
        columns = [col_map.get(h, h) for h in headers]
        placeholders = ",".join(["%s"] * len(columns))
        col_str = ",".join(columns)
        sql = f"INSERT IGNORE INTO ({table_name}) ({col_str}) VALUES ({placeholders})"
        rows = []
        for row in reader:
            cleaned = []
            for val in row:
                if val.strip() in ("NULL", ""):
                    cleaned.append(None)
                else:
                    cleaned.append(val.strip())
            rows.append(cleaned)
        cursor.executemany(sql, rows)

def main():
    conn = mysql.connector.connect(**DB_CONFIG)
    cursor = conn.cursor()
    cursor.execute("DROP DATABASE IF EXISTS {DATABASE}")
    cursor.execute("CREATE DATABASE {DATABASE}")
    cursor.execute("USE {DATABASE}")

    for stmt in extract_create_statements(SQL_FILE):
        cursor.execute(stmt)
    print("Schema created.")

    cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
    for csv_file in AD_ORDER:
        table = TABLE_MAP.get(csv_file, csv_file.replace(".csv", ""))
        col_map = COLUMN_MAP.get(csv_file, {})
        load_csv(cursor, csv_file, table, col_map)
        conn.commit()
        print(f"Loaded {csv_file} => {table}")
    cursor.execute("SET FOREIGN_KEY_CHECKS = 1")

    print("\nHow counts:")
    for csv_file in AD_ORDER:
        table = TABLE_MAP.get(csv_file, csv_file.replace(".csv", ""))
        cursor.execute("SELECT COUNT(*) FROM {table}")
        count = cursor.fetchone()[0]
        print(f"{table}: {count}")

    cursor.close()
    conn.close()

if __name__ == "__main__":
    main()
```

Query - Mult. Tables/Grouping

Query 1: Total revenue generated by each route, ordered from highest to lowest.

```
SELECT orig.airport_code AS origin, dest.airport_code AS destination,  
       SUM(p.amount_paid) AS total_revenue  
FROM payment p JOIN booking b JOIN booking_flight bf  
      JOIN flight f JOIN route r JOIN airport orig JOIN airport dest  
ON p.booking_id = b.booking_id AND b.booking_id = bf.booking_id  
   AND bf.flight_id = f.flight_id AND f.route_id = r.route_id  
   AND r.origin_airport_id = orig.airport_id  
   AND r.destination_airport_id = dest.airport_id  
WHERE p.payment_status = 'Paid'  
GROUP BY orig.airport_code, dest.airport_code  
ORDER BY total_revenue DESC;
```

Query Result

Query 1: Total revenue generated by each route, ordered from highest to lowest.

origin	destination	total_revenue
JFK	HNL	33727.42
LAX	MIA	21349.94
SFO	ORD	20873.20
ATL	MIA	19549.31
LAX	SEA	18846.26
LAX	JFK	16500.91
SEA	DEN	16134.02
JFK	MCO	15411.70
SJC	PDX	14194.37
AUS	MCO	13611.64
SFO	DFW	13567.37
LAX	ORD	13457.95
PDX	HNL	12817.49
BOS	MCO	12744.17
SFO	DEN	12411.46
ORD	ATL	10739.48
SFO	SAN	10244.95
DEN	MSP	9945.66
ORD	MSP	9885.99
LAX	LAS	9809.27
ORD	DFW	8641.53
ORD	DEN	8450.71
DEN	SLC	8241.97
MSP	SLC	7631.14
LAX	DEN	7594.12

Query - Mult. Tables/Grouping

Query 2: Which pilot has flown the most flights, ranked from highest to lowest.

```
SELECT p.first_name, p.last_name, ep.employee_number,  
       a.airline_name, COUNT(f.flight_id) AS flights_flown  
FROM   employee_passenger ep  
       JOIN passenger p  
       JOIN airline a  
       JOIN flight f  
ON     ep.passenger_id = p.passenger_id  
       AND ep.airline_id = a.airline_id  
       AND f.airline_id = a.airline_id  
WHERE  ep.job_title = 'Pilot'  
GROUP BY ep.passenger_id, p.first_name,  
         p.last_name, ep.employee_number, a.airline_name  
ORDER BY flights_flown DESC;
```


Query Result

Query 2: Which pilot has flown the most flights, ranked from highest to lowest.

first_name	last_name	employee_number	airline_name	flights_flown
Marcus	Webb	EMP-UA-0001	United Airlines	31
James	Carter	EMP-DL-0001	Delta Air Lines	29
Brian	Tran	EMP-WN-0001	Southwest Airlines	27

Implementations - Queries

Query 3: Which flight has the highest occupancy?

```
#Query One
SELECT f.flight_id, Count(passenger_id)
FROM booking_flight bf JOIN flight f
ON bf.flight_id = f.flight_id
GROUP BY flight_id
ORDER BY count(passenger_id) DESC
LIMIT 1;
```

flight_...	Count(passenger_...
64	10

Query 4: Average age of passengers per flight?

```
#Query Two
SELECT f.flight_id, f.departure_time,
AVG(YEAR(CURDATE()) - YEAR(ap.date_of_birth)) as avg_age
FROM flight f
JOIN booking_flight bf ON f.flight_id = bf.flight_id
JOIN passenger p ON bf.passenger_id = p.passenger_id
JOIN adult_passenger ap ON p.passenger_id = ap.passenger_id
GROUP BY f.flight_id, f.departure_time
ORDER BY f.flight_id
LIMIT 10;
```

flight_...	departure_time	avg_age
1	2026-06-01 12:15:00	37.4000
2	2026-06-01 18:00:00	38.1667
3	2026-06-01 12:45:00	41.0000
4	2026-06-01 10:45:00	46.3333
5	2026-06-01 08:00:00	36.3333
6	2026-06-02 08:45:00	45.5000
7	2026-06-02 12:00:00	45.8000
8	2026-06-02 08:15:00	40.0000
9	2026-06-02 16:15:00	47.7500
10	2026-06-02 08:15:00	39.0000

Implementations - Queries

Query 5: What is the first and last name of the passengers who sit in seat 14E?

```
SELECT passenger_id, first_name, last_name
FROM passenger
WHERE passenger_id IN(SELECT passenger_id
FROM booking_flight bf JOIN seat_assignment sa
ON bf.booking_flight_id = sa.booking_flight_id
WHERE seat_number = "14E")
ORDER BY passenger_id;
```

passenger_id	first_name	last_name
1	Alexander	Bailey
2	Ronald	Cox
3	Barbara	Jones
4	Megan	Gutierrez
5	Emily	Adams
9	Michael	Kim
12	Edward	Williams

Query 6: List passengers alphabetically from flight 27

```
SELECT p.passenger_id, p.first_name, p.last_name
FROM passenger p JOIN booking_flight bf
ON p.passenger_id = bf.passenger_id
WHERE flight_id = "27"
ORDER BY last_name ASC, first_name ASC;
```

passenger_id	first_name	last_name
11	Heather	Jackson
3	Barbara	Jones
7	Camila	Price

Implementations - Queries

Query 7: Which flights have a higher average ticket price than the overall average?

```
SELECT f.flight_id, AVG(bf.ticket_price)
FROM flight f JOIN booking_flight bf
ON f.flight_id = bf.flight_id
GROUP BY flight_id
HAVING AVG(ticket_price) > (SELECT AVG(ticket_price)
FROM booking_flight)
ORDER BY AVG(bf.ticket_price)
LIMIT 10;
```

flight_...	AVG(bf.ticket_pri...
53	236.180000
163	236.801250
32	238.853333
245	238.863333
168	238.998000
192	239.253333
179	239.946667
216	241.275000
15	241.402500
97	241.443333

Discussion - Takeaways

Potential scenarios where this database could be useful:

1. Determining pilots experience measured in the number of flights they have flown
2. Identifying low demand routes to cut to save airline costs
3. Distinguishing the most profitable routes

Key Takeaways

- Building an ER database starting from the conceptual design and transferring it into SQL taught us how important a good ER diagram and logical design is.
- Well designed databases and queries are crucial to the smooth flow of information and operation between a booking website and an airport.